BACKEND READING DOCUMENTATION

Books Library Backend Code That Makes Books Readable

شرح كود الخلفية الذي يجعل الكتب قابلة للقراءة والاستماع

This document explains only the backend code responsible for turning catalog entries into readable books, serving chapters and pages, and generating chapter audio. It focuses on the full reading pipeline, not the frontend screens.

يشرح هذا المستند فقط كود الخلفية المسؤول عن تحويل عناصر الفهرس إلى كتب قابلة للقراءة، وتقديم الفصول والصفحات، وتوليد صوت الفصول. ويركز على مسار القراءة الكامل داخل الخلفية، وليس على شاشات الواجهة.

4	6	SQL	TTS
Main backend reading stages: import, schema, serving text, serving audio.	Core backend files explained in depth.	Books, chapters, pages, and audio cache tables.	ElevenLabs first, gTTS fallback, cached MP3 output.

Prepared on 14 April 2026 as a backend-only explanation companion for the project.

SCOPE

1. What This Backend Is Responsible For

The backend does not only store book metadata. It creates the database schema, imports readable content, answers API requests for books/chapters/pages, and builds playable audio for each chapter.

English summary

The reading backend has one main job: move book content from raw source files into MySQL tables, then expose that content through FastAPI endpoints so the app can read and listen to it.

الملخص بالعربية

المهمة الأساسية لهذه الخلفية هي نقل محتوى الكتب من ملفات المصدر الخام إلى جداول MySQL، ثم تقديم هذا المحتوى عبر واجهات FastAPI لكي يستطيع التطبيق قراءته والاستماع إليه.

Reading pipeline

1. Create the database tables
2. Insert catalog books
3. Import readable text into `chapters` and `pages`
4. Expose chapters and pages through API routes
5. Generate chapter audio when requested

مسار القراءة

1. إنشاء جداول قاعدة البيانات
2. إدخال كتب الفهرس
3. استيراد النص المقروء إلى `chapters` و`pages`
4. تقديم الفصول والصفحات عبر الواجهات البرمجية
5. توليد صوت الفصل عند الطلب

> In this project, a book becomes “readable” only when it has rows inside the `chapters` table and matching rows inside the `pages` table.

DATABASE ENTRY

2. Database Connection and Why It Matters

Everything starts with the connection file because every reading feature depends on MySQL.

`book_backend/database.py`

`Lines 1-10`

`get_connection()`

English: This function creates the MySQL connection object using the host, username, password, and database name. The rest of the backend never opens a database manually; it asks this helper for a connection.

العربية: تقوم هذه الدالة بإنشاء اتصال MySQL باستخدام عنوان الخادم واسم المستخدم وكلمة المرور واسم قاعدة البيانات. بقية ملفات الخلفية لا تنشئ الاتصال مباشرة، بل تطلبه من هذا المساعد.

- Why important: all reading APIs depend on this function
- Used by: `main.py`, `catalog_seed.py`, `import_book.py`, `import_amwaj_akma.py`

`book_backend/database.py`

`Lines 1-10`

فكرة الملف

هذا الملف صغير جداً، لكنه أساسي، لأن أي مشكلة في الاتصال بقاعدة البيانات ستمنع قراءة الكتب وجلب الفصول والصفحات وتخزين الصوت.

CORE DEPENDENCY

SCHEMA

3. How The Backend Creates Reading Tables

The next important layer is schema creation, because readable books need structure before data can be imported.

`book_backend/catalog_utils.py`

`Lines 14-70`

`ensure_catalog_schema(curs

English: This function creates the main reading tables if they do not exist: `books`, `chapters`, and `pages`. It also upgrades older versions of the `books` table by adding `language`, `is_featured`, and `sort_order` if those columns are missing.

العربية: تنشئ هذه الدالة الجداول الأساسية للقراءة إذا لم تكن موجودة: `books` و`chapters` و`pages`. كما تقوم بتحديث الإصدارات القديمة من جدول `books` بإضافة الأعمدة `language` و`is_featured` و`sort_order` إذا كانت مفقودة.

`book_backend/catalog_utils.py`

`Lines 73-107`

`ensure_audio_table(cursor)`

English: Creates the `chapter_audio` table and guarantees there is a unique combination of `(chapter_id, voice_name)`. This means the same chapter can store separate audio files for Rachel and Adam.

العربية: تنشئ هذه الدالة جدول `chapter_audio` وتضمن وجود قيد فريد على `(chapter_id, voice_name)`. وهذا يعني أن الفصل نفسه يمكن أن يملك ملفي صوت منفصلين لـ Rachel وAdam.

`book_backend/catalog_utils.py`

`Lines 116-155`

`clear_existing_book_content and replace_book_content()`

English: These two functions are what really make a book readable. First the old chapter/page/audio rows for a book are deleted. Then new chapters are inserted into `chapters`, and every paragraph/page is inserted into `pages`.

العربية: هاتان الدالتان هما ما يجعل الكتاب قابلاً للقراءة فعلياً. إذ يتم أولاً حذف الصفوف القديمة الخاصة بالفصول والصفحات والصوت، ثم إدخال الفصول الجديدة إلى `chapters`، وإدخال كل فقرة أو صفحة إلى `pages`.

`book_backend/catalog_utils.py`

`Lines 158-168`

`()reset_catalog_tables`

تقوم هذه الدالة بمسح الجداول وإعادة ترقيمها عند بذر الفهرس من جديد. وهي مهمة في بيئة العرض والتجربة لأنها تضمن بدء النظام من حالة نظيفة ومعلومة.

> Readability is not a boolean stored manually. The API later derives it from the existence of chapter and page rows.

IMPORT

4. How Raw Book Files Become Readable Books

These scripts are the bridge between source files and the database used by the app.

`book_backend/import_book.py`

`Lines 22-66`

English book normalization

English: `repair_text()` fixes broken text encoding. `normalize_chapters()` cleans titles and paragraphs, then rebuilds the chapter list into a structured format the database can accept.

العربية: تقوم `repair_text()` بإصلاح الترميز المكسور. ثم تنظف `normalize_chapters()` العناوين والفقرات، وتعيد بناء قائمة الفصول في صيغة منظمة يمكن لقاعدة البيانات استقبالها.

`book_backend/import_book.py`

`Lines 69-118`

English book database write

English: `upsert_book()` creates or updates the main book row. `import_book()` reads the JSON file, normalizes chapters, ensures the schema exists, and passes the result to `replace_book_content()` so the book becomes readable.

العربية: تنشئ `upsert_book()` سجل الكتاب أو تحدثه. ثم تقرأ `import_book()` ملف JSON، وتنظم الفصول، وتضمن وجود الجداول، ثم تمرر النتيجة إلى `replace_book_content()` لكي يصبح الكتاب قابلاً للقراءة.

`book_backend/import_amwaj_akma.py`

`Lines 79-128`

Arabic page cleaning and section building

English: `is_front_matter()` removes non-book opening material. `clean_page_text()` strips links, Latin noise, repeated punctuation, and number artifacts. `build_sections()` groups pages into chapter-like sections.

العربية: تزيل `is_front_matter()` المادة الافتتاحية غير الخاصة بالنص. وتحذف `clean_page_text()` الروابط والضجيج اللاتيني والتكرارات وعلامات الأرقام. ثم تجمع `build_sections()` الصفحات في أقسام تشبه الفصول.

`book_backend/import_amwaj_akma.py`

`Lines 131-192`

Arabic conversion and import

English: `convert_book()` turns the raw Arabic source file into structured chapters and also saves a generated JSON copy. Then `import_book()` writes those sections into MySQL through `replace_book_content()`.

العربية: تحول `convert_book()` ملف المصدر العربي الخام إلى فصول منظمة، وتحفظ أيضاً نسخة JSON مولدة. ثم تقوم `import_book()` بكتابة هذه الأقسام داخل MySQL عبر `replace_book_content()`.

`book_backend/catalog_seed.py`

`Lines 92-117`

Full seed process

English: `seed_catalog()` resets the database, inserts all catalog books, then imports the two books that contain real readable content. This is why only those titles are readable inside the app even though the catalog is larger.

العربية: تقوم `seed_catalog()` بإعادة ضبط قاعدة البيانات، ثم تدخل جميع كتب الفهرس، وبعد ذلك تستورد الكتابين اللذين يحتويان على محتوى مقروء فعلياً. ولهذا السبب لا تكون كل الكتب قابلة للقراءة داخل التطبيق رغم أن الفهرس أكبر.

API

5. How The Backend Serves Readable Text

Now that content exists in MySQL, `main.py` exposes it to the Flutter app.

`book_backend/main.py`

`Lines 49-66`

App startup and schema safety

English: `app = FastAPI()` creates the server, CORS is enabled, `/media` is mounted for audio files, and `startup_event()` ensures the schema exists every time the backend starts.

العربية: يقوم `app = FastAPI()` بإنشاء الخادم، ويتم تفعيل CORS، وربط `/media` لملفات الصوت، كما تتأكد `startup_event()` من وجود الجداول في كل مرة يبدأ فيها الخادم.

`book_backend/main.py`

`Lines 291-337`

`GET /books`

English: Returns the book catalog. The important part is the SQL query computes `chapter_count`, `page_count`, and `is_readable` by joining `books`, `chapters`, and `pages`. So the frontend does not guess readability; the backend calculates it.

العربية: تعيد هذه الواجهة فهرس الكتب. والجزء المهم فيها أن استعلام SQL يحسب `chapter_count` و`page_count` و`is_readable` عبر الربط بين `books` و`chapters` و`pages`. لذلك لا تخمن الواجهة حالة القراءة، بل تحسبها الخلفية.

`book_backend/main.py`

`Lines 340-366`

`GET /books/summary`

English: Produces dashboard totals for all books, readable books, English books, and Arabic books. It is mainly for the home screen statistics, but it also proves the backend knows the real state of the library.

العربية: تنتج هذه الواجهة الإحصاءات العامة لجميع الكتب والكتب القابلة للقراءة والإنجليزية والعربية. وهي مخصصة لبطاقات الصفحة الرئيسية، لكنها تثبت أيضاً أن الخلفية تعرف الحالة الحقيقية للمكتبة.

`book_backend/main.py`

`Lines 369-396`

`GET /books/{book_id}/chapters` and `GET /chapters/{chapter_id}/pages`

English: These are the two endpoints that directly make reading possible. The first returns ordered chapter records for one book; the second returns ordered page content for one chapter.

العربية: هاتان الواجهتان هما اللتان تجعلان القراءة ممكنة مباشرة. فالأولى تعيد الفصول المرتبة لكتاب واحد، والثانية تعيد الصفحات المرتبة الخاصة بفصل واحد.

AUDIO

6. How The Backend Generates Chapter Audio

The audio pipeline is the most advanced reading-related backend feature in the project.

`book_backend/main.py`

`Lines 69-127`

Loading chapter text and building narration text

English: `get_chapter_with_pages()` loads chapter metadata and all page rows. `build_audio_text()` combines the book title, chapter number, chapter title, and page content into one narration script. It also decides whether the intro should be Arabic or English.

العربية: تقوم `get_chapter_with_pages()` بتحميل بيانات الفصل وجميع صفحاته. ثم تجمع `build_audio_text()` اسم الكتاب ورقم الفصل وعنوانه ومحتوى الصفحات في نص سرد واحد، وتحدد أيضاً إن كانت المقدمة يجب أن تكون بالعربية أو الإنجليزية.

`book_backend/main.py`

`Lines 130-203`

Text cleanup and chunking

English: `normalize_tts_text()` removes markup artifacts and bad punctuation for speech. `detect_tts_language()` uses Arabic character count to choose the language. `split_text_for_tts()` breaks long text into safe chunks so the TTS providers can handle it.

العربية: تحذف `normalize_tts_text()` آثار التنسيق والعلامات غير المناسبة للكلام. وتستخدم `detect_tts_language()` عدد الحروف العربية لتحديد اللغة. ثم تقسم `split_text_for_tts()` النص الطويل إلى أجزاء آمنة ليتمكن مزود الصوت من معالجتها.

`book_backend/main.py`

`Lines 206-288`

Audio providers and fallback strategy

English: `strip_leading_id3_tag()` prevents duplicate MP3 headers when chunks are merged. `get_elevenlabs_client()` loads the premium provider if configured. `save_audio_with_elevenlabs()` and `save_audio_with_gtts()` create MP3 files, while `save_audio_with_fallback()` tries ElevenLabs first and falls back to gTTS if needed.

العربية: `strip_leading_id3_tag()` تمنع تكرار ترويسات MP3 عند دمج الأجزاء. وتحمّل `get_elevenlabs_client()` المزود المتقدم إذا كان مضبوطاً. ثم تنشئ `save_audio_with_elevenlabs()` و`save_audio_with_gtts()` الملفات الصوتية، بينما تحاول `save_audio_with_fallback()` استخدام ElevenLabs أولاً ثم تنتقل إلى gTTS عند الحاجة.

`book_backend/main.py`

`Lines 399-470`

`GET /chapters/{chapter_id}/audio`

English: This route is the final audio endpoint. It checks whether chapter content exists, builds a file name based on book/chapter/voice, looks for a cached audio row, generates audio when missing, writes the path into `chapter_audio`, and finally returns a complete public media URL.

العربية: تمثل هذه الواجهة النهاية الفعلية لمسار الصوت. فهي تتحقق من وجود المحتوى، ثم تبني اسم الملف حسب الكتاب والفصل والصوت، وتبحث عن نسخة مخزنة في `chapter_audio`، وتولد الصوت إذا لم يوجد، ثم تحفظ المسار، وأخيراً تعيد رابطاً كاملاً قابلاً للتشغيل.

> The first audio request may be slow because the backend is generating the file. Later requests become faster because the route can return cached audio.

EXECUTION PATH

7. Full Backend Reading Flow From Source File To App Screen

This is the clearest way to explain the complete backend code during presentation.

Step 1: raw source file exists

Files: `import_book.py` or `import_amwaj_akma.py`

English: The project begins with a raw JSON or page source file that still needs cleanup.

العربية: يبدأ المشروع من ملف مصدر خام بصيغة JSON أو صفحات نصية تحتاج إلى تنظيف وتنظيم.

Step 2: source is cleaned and normalized

Files: `import_book.py`, `import_amwaj_akma.py`

English: The import code repairs encoding, removes noise, groups content into chapters or sections, and prepares a structured list.

العربية: يقوم كود الاستيراد بإصلاح الترميز وحذف الضجيج وتجميع المحتوى في فصول أو أقسام ثم إعداد قائمة منظمة.

Step 3: tables are guaranteed to exist

Files: `catalog_utils.py`, `main.py`

English: The backend ensures that the schema exists before writing or serving anything.

العربية: تتأكد الخلفية من وجود الجداول قبل كتابة أي شيء أو تقديمه للتطبيق.

Step 4: readable content is written

Files: `catalog_utils.py`

English: The content is inserted into `chapters` and `pages`, which is the real moment when a book becomes readable in the app.

العربية: يتم إدخال المحتوى في `chapters` و`pages`، وهذه هي اللحظة الحقيقية التي يصبح فيها الكتاب قابلاً للقراءة داخل التطبيق.

Step 5: the frontend requests chapters/pages

Files: `main.py` endpoints

English: The backend returns ordered chapter and page rows so the app can display the reading UI.

العربية: تعيد الخلفية صفوف الفصول والصفحات مرتبة حتى تتمكن الواجهة من بناء شاشة القراءة.

Step 6: audio is generated on demand

Files: `main.py`, `chapter_audio` table

English: The backend transforms chapter text into speech, stores the MP3 path, and returns a URL to the app.

العربية: تحول الخلفية نص الفصل إلى كلام، وتحفظ مسار MP3، ثم تعيد للرابط إلى التطبيق.

KEY POINTS

8. What To Say About The Full Backend Code

These are the strongest technical ideas inside this backend.

Readable books are data-driven

The backend does not mark a book readable by hand in the API layer. A book becomes readable because chapter and page rows were imported into the database.

قابلية القراءة مبنية على البيانات

لا تقوم الخلفية بتحديد الكتاب كقابل للقراءة يدوياً داخل الواجهة البرمجية، بل يصبح الكتاب قابلاً للقراءة لأن الفصول والصفحات تم استيرادها فعلاً إلى قاعدة البيانات.

Schema creation is built into runtime

The server guarantees the required tables at startup, which makes demo deployment and testing easier.

إنشاء الجداول جزء من التشغيل

يضمن الخادم وجود الجداول عند الإقلاع، وهذا يسهل العرض والتجربة والاختبار في بيئة المشروع.

Audio is a real backend service

The audio feature is not just a frontend button. It includes text preparation, language detection, chunking, provider selection, cache control, file storage, and public URL generation.

الصوت خدمة خلفية حقيقية

ميزة الصوت ليست مجرد زر في الواجهة، بل تشمل تجهيز النص، وتحديد اللغة، وتقسيم المحتوى، واختيار مزود الصوت، والتحكم بالكاش، وتخزين الملف، وإنشاء الرابط النهائي.

Import scripts are part of the product

The book import scripts are essential, because without them the app would have catalog entries but no full readable content.

سكربتات الاستيراد جزء من المنتج

تعد سكربتات الاستيراد جزءاً أساسياً، لأنه من دونها سيبقى التطبيق مجرد فهرس كتب بلا محتوى كامل قابل للقراءة.

Final summary: the backend makes reading possible by creating the schema, importing structured content into chapters/pages, serving those rows through API endpoints, and generating cached audio from the same text.